

Preconditioning L-BFGS with the Analytic FEM Stiffness

Theory and implementation notes for the MTM lattice-triangulation code

Technical note

September 2026

Abstract

The quasi-static shear simulations of the MTM code spend almost all of their time (97% in a 150×150 production run) minimizing the energy on the *elastic branch*, i.e. in load steps where nothing dramatic happens. Plain L-BFGS needs $\sim 10^3$ iterations per such step, and this number grows with the system size. This note explains, from the ground up, why that happens and how we fixed it: we build the exact analytic FEM stiffness matrix K from the second derivatives that are already coded in the energy functions, factorize it with a sparse Cholesky decomposition once per load step, and feed it to ALGLIB's L-BFGS through a linear change of variables. ALGLIB itself is not modified. On elastic steps the number of iterations drops from ~ 1250 to ~ 37 at 150×150 , and the wall time per step from 5.5 s to 0.37 s (about $15\times$). Along the way we found and fixed a factor-of-two inconsistency between the energy and its gradient. Everything is opt-in (`--precond=stiffness`); the default behaviour of the code is unchanged. The same factorization machinery also computes the lowest eigenvalues of K (stability analysis) $9\text{--}60\times$ faster than before (Section 12). Section 13 and the annotated bibliography place all this in the literature.

Contents

1	The problem we are trying to solve	3
2	The discrete model	3
2.1	Kinematics	3
2.2	Energy and Lagrange reduction	3
2.3	Unknowns	4
3	First derivatives: stress and forces	4
3.1	The tensor derivative convention	4
3.2	First Piola–Kirchhoff stress	4
3.3	Nodal forces	5
3.4	A factor of two we found on the way	5
4	Why plain L-BFGS is slow here	5
4.1	L-BFGS in one page	5
4.2	Condition number of an elastic body	6
4.3	What ALGLIB offers, and why it is not enough	6
5	The key idea: preconditioning by a change of variables	7
5.1	The transformation	7
5.2	This is exactly preconditioned L-BFGS	7
5.3	Which P ?	8

6	The analytic stiffness matrix	8
6.1	Second derivatives of φ with respect to the metric	8
6.2	The Lagrangian tangent	9
6.3	From the tangent to the element stiffness	9
6.4	Getting the second derivatives without ITensor	10
6.5	Global assembly	10
6.6	Validation of K	10
7	Turning K into a preconditioner	11
7.1	Indefiniteness: element-wise PSD projection	11
7.2	Rigid translations: a small shift	11
8	Sparse Cholesky and the two half-solves	11
8.1	Factorization	11
8.2	Applying R^{-1} and R^{-T}	12
9	The algorithm as implemented	12
9.1	One call of <code>PreconditionedLBFGS::optimize</code>	12
9.2	Stopping criterion	13
9.3	Restarts when the preconditioner goes stale	13
9.4	Refresh policy	13
9.5	The Laplacian variant	13
9.6	Cost of one elastic step at 150×150	13
10	Results	14
10.1	Elastic steps	14
10.2	Avalanches	14
10.3	The production binary	14
11	Physics caveats: path dependence	15
12	Stability analysis with the same machinery: the lowest eigenvalues of K	15
12.1	What we want and how it was done before	15
12.2	Shift-invert Lanczos in one paragraph	16
12.3	Three improvements	16
12.4	Results	17
12.5	Monitoring stability during the run	18
12.6	Saddle-node analysis	18
13	Related work: what is known and what is new	19
14	Implementation map	20
15	Where the remaining time goes, and possible next steps	21

1 The problem we are trying to solve

Let me start with the physics workflow, because the whole story is about *where* the time goes.

A simulation (e.g. `example_1_conti_zanzotto_loading`) applies simple shear

$$F_{\text{ext}}(\alpha) = \begin{pmatrix} 1 & \alpha \\ 0 & 1 \end{pmatrix}, \quad \alpha = 0.14, 0.14006, 0.14012, \dots, 1.0, \quad (1)$$

in about 14 334 small increments $\Delta\alpha = 6 \times 10^{-5}$. At every load step the nodes are first mapped affinely by the increment, and then the total energy is *minimized* over the node positions with L-BFGS (ALGLIB `minlbfgs`). This is the classical quasi-static protocol: the system is assumed to follow a local minimum of the energy, and when that minimum disappears (an instability) the minimizer carries the system to a new one. Those events are the avalanches / plastic events we are interested in.

The key observation is statistical: *avalanches are rare*. In the reference 150×150 run (`run_zanzotto_loading_150x150`, seed 42):

	steps	time in the main minimization
elastic steps (no remeshing)	14 153 (98.7%)	11.3 h
remeshing / avalanche steps	180 (1.3%)	0.4 h

So 97% of the 11.7 h wall time is spent solving *boring* problems: a lattice that is already in equilibrium receives a tiny homogeneous shear and has to relax a tiny bit. Each of those relaxations took on average ~ 2.9 s and $\sim 10^3$ L-BFGS iterations. That is absurdly expensive for what is essentially a linear-elastic response, and it is precisely the regime where second-order information (the Hessian) should help enormously.

2 The discrete model

2.1 Kinematics

The reference lattice is triangulated into P1 (linear) triangles. For an element e with nodes $a = 0, 1, 2$, current positions $\mathbf{x}_a \in \mathbb{R}^2$ and constant shape function gradients $\nabla N_a = (\partial N_a / \partial X_1, \partial N_a / \partial X_2)$, the deformation gradient is constant on the element:

$$F = \sum_{a=0}^2 \mathbf{x}_a \otimes \nabla N_a \quad \Longleftrightarrow \quad F_{iK} = \sum_a x_{a,i} \frac{\partial N_a}{\partial X_K}. \quad (2)$$

In the code this is `F = x_e * dNdX` with `x_e` the 2×3 matrix of node positions and `dNdX` the 3×2 matrix of shape-function gradients (`ElementTriangle2D::calculate_deformation_gradient`). With periodic boundary conditions a node of an element may be a periodic image; its position is $\mathbf{x}_{\text{orig}} + F_{\text{ext}} \mathbf{t}$ with a fixed lattice translation $\mathbf{t}$. The translation does not depend on the unknowns, so for all derivatives below a periodic image behaves exactly like its original node.

The right Cauchy–Green (metric) tensor is $C = F^\top F$.

2.2 Energy and Lagrange reduction

The strain energy density is the Conti–Zanzotto type function $\varphi(C)$ of the metric, invariant under the lattice symmetry group $GL(2, \mathbb{Z})$. It is evaluated on the *reduced* metric

$$C_{\text{red}} = Z^\top C Z = (FZ)^\top (FZ), \quad Z \in GL(2, \mathbb{Z}), \quad (3)$$

where Z (`m_matrix`) is produced by `lagrange::reduce(C)` and brings C into the fundamental domain. Z is piecewise constant: it only jumps when C crosses the boundary of the fundamental domain, and because φ is $GL(2, \mathbb{Z})$ -invariant the energy is continuous across such a jump. Inside each cell of the reduction we can therefore differentiate with Z held fixed. This is what the gradient code does, and what the Hessian code must do too.

The total energy is

$$E(\mathbf{x}) = \sum_e A_e \frac{\varphi(C_{\text{red}}^{(e)}) - \varphi_0}{a^2}, \quad (4)$$

with A_e the reference area (`getReferenceArea()`), a the lattice parameter and φ_0 the energy of the undeformed crystal.

2.3 Unknowns

The free nodes are numbered $0, \dots, n-1$ through `full_mapping[node].second`, and the solver vector is

$$\mathbf{x} = (u_0, \dots, u_{n-1}, v_0, \dots, v_{n-1}) \in \mathbb{R}^{2n}, \quad (5)$$

i.e. first all x -components, then all y -components. At 150×150 with PBC, $n = 22\,500$ and there are 45 000 unknowns; there are 45 000 elements.

3 First derivatives: stress and forces

3.1 The tensor derivative convention

φ is coded as a function of three scalars (c_{11}, c_{22}, c_{12}) . A symmetric matrix has four entries but only three independent ones, so we have to be careful what “ $\partial\varphi/\partial C$ ” means. We define the tensor derivative D by requiring

$$d\varphi = \sum_{a,b} D_{ab} dC_{ab} \quad \text{for all symmetric } dC. \quad (6)$$

Writing it out, $d\varphi = \varphi_{,11} dc_{11} + \varphi_{,22} dc_{22} + \varphi_{,12} dc_{12}$ while $\sum D_{ab} dC_{ab} = D_{11} dc_{11} + D_{22} dc_{22} + (D_{12} + D_{21}) dc_{12}$, hence

$$D_{11} = \varphi_{,11}, \quad D_{22} = \varphi_{,22}, \quad D_{12} = D_{21} = \frac{1}{2} \varphi_{,12}. \quad (7)$$

This is exactly what `dphi_func` returns (it computes $\varphi_{,12}$ and then divides the off-diagonal entry by two: “we go from DE/DC to J”). I checked it against finite differences of `phi_func`: all three entries agree to 10^{-9} .

3.2 First Piola–Kirchhoff stress

Set $G = FZ$ so that $C_{\text{red}} = G^T G$, i.e. $(C_{\text{red}})_{ab} = G_{ia} G_{ib}$ (summation over repeated indices). Since $G_{ia} = F_{iM} Z_{Ma}$, we have $\partial G_{ia} / \partial F_{jK} = \delta_{ij} Z_{Ka}$ and

$$B_{ab,jK} := \frac{\partial (C_{\text{red}})_{ab}}{\partial F_{jK}} = Z_{Ka} G_{jb} + Z_{Kb} G_{ja}. \quad (8)$$

The stress is then (per unit reference area, dropping $1/a^2$ for readability)

$$P_{jK} = \frac{\partial \varphi}{\partial F_{jK}} = D_{ab} B_{ab,jK} = (GD^T Z^T)_{jK} + (GDZ^T)_{jK} = 2(FZDZ^T)_{jK}, \quad (9)$$

which is the line in `minimize_energy_with_triangles`:

```
const Eigen::Matrix2d P = 2.0 * F * result.m_matrix * dE_dC
                        * result.m_matrix.transpose();
```

It is also useful to name the second Piola–Kirchhoff stress

$$S = 2 Z D Z^\top, \quad P = F S. \quad (10)$$

3.3 Nodal forces

From (2), $\partial F_{iK}/\partial x_{a,j} = \delta_{ij} \partial N_a/\partial X_K$, so the contribution of element e to the gradient is

$$\frac{\partial E_e}{\partial \mathbf{x}_a} = A_e P \nabla N_a^\top. \quad (11)$$

This is `ElementTriangle2D::calculate_nodal_forces: (P * dNdX.row(a).transpose()) * area.`

3.4 A factor of two we found on the way

Until this work, the line computing P in `minimize_energy_with_triangles` ended with `*` `area`, and `calculate_nodal_forces` multiplied by `area` again. So the code returned $A_e \partial E/\partial \mathbf{x}$ instead of $\partial E/\partial \mathbf{x}$.

Why did nobody notice? Because on the square lattice every reference triangle has area exactly $A_e = \frac{1}{2}$. (This is even true after remeshing: a Delaunay triangle of the lattice contains no other lattice points, and by Pick’s theorem such a lattice triangle has area $\frac{1}{2}$.) So the gradient was exactly $\frac{1}{2} \nabla E$ everywhere: the zeros of the gradient, i.e. the equilibria, were correct, and all the physics was fine. What was *not* fine is that L-BFGS’s line search uses E and $\mathbf{g}$ together (sufficient decrease and curvature conditions, cubic interpolation), and they disagreed by a factor 2.

We discovered this while validating the Hessian: the directional check $K\mathbf{v} \approx [\mathbf{g}(\mathbf{x}+\epsilon\mathbf{v}) - \mathbf{g}(\mathbf{x}-\epsilon\mathbf{v})]/2\epsilon$ came out as 0.500000 for every random direction, and an energy check $dE/dv \approx 2\mathbf{g} \cdot \mathbf{v}$ told us which side was wrong. After removing the extra area (only in `minimize_energy_with_triangles`; the `_noredution` variant was left alone on purpose) both checks agree to 10^{-7} , the trajectory of the default solver is unchanged to 5×10^{-10} in energy, and the plain solver itself became about 16% faster (fewer line-search evaluations).

4 Why plain L-BFGS is slow here

4.1 L-BFGS in one page

L-BFGS is a quasi-Newton method. At iteration k it moves along

$$\mathbf{d}_k = -H_k \mathbf{g}_k, \quad (12)$$

where $H_k \approx (\nabla^2 E)^{-1}$ is an approximation of the inverse Hessian built from the last m pairs (ALGLIB with $m = 13$ in the code)

$$\mathbf{s}_i = \mathbf{x}_{i+1} - \mathbf{x}_i, \quad \mathbf{y}_i = \mathbf{g}_{i+1} - \mathbf{g}_i. \quad (13)$$

H_k is never formed; the product $H_k \mathbf{g}_k$ is computed with the famous *two-loop recursion*:

```
q ← g_k
for i ← m-1 to 0
  ρ_i = 1/(y_i^T s_i)
  r ← H_0 q
  for j ← i-1 to 0
    r ← r + (θ_j - ρ_j) s_j
  d_k = -r
```

The *initial* inverse Hessian H_0 is the only free choice. By default ALGLIB uses a multiple of the identity,

$$H_0 = \gamma_k I, \quad \gamma_k = \frac{\mathbf{s}_{k-1}^\top \mathbf{y}_{k-1}}{\mathbf{y}_{k-1}^\top \mathbf{y}_{k-1}}, \quad (14)$$

which only fixes the overall scale. The step length is then determined by a Moré–Thuente line search.

4.2 Condition number of an elastic body

Near a stable equilibrium the energy is essentially quadratic, $E \approx \frac{1}{2}(\mathbf{x} - \mathbf{x}^*)^\top K(\mathbf{x} - \mathbf{x}^*)$, where $K = \nabla^2 E$ is the FEM stiffness matrix. For linear elasticity discretized with P1 elements on an $L \times L$ grid, K behaves like a discrete Laplacian. For the 5-point Laplacian with periodic boundary conditions the eigenvalues are

$$\lambda(k_1, k_2) = 4 \sin^2 \frac{\pi k_1}{L} + 4 \sin^2 \frac{\pi k_2}{L}, \quad k_1, k_2 = 0, \dots, L-1, \quad (15)$$

so (ignoring the zero mode, a rigid translation)

$$\lambda_{\max} \approx 8, \quad \lambda_{\min} \approx 4 \sin^2 \frac{\pi}{L} \approx \frac{4\pi^2}{L^2}, \quad \kappa(K) = \frac{\lambda_{\max}}{\lambda_{\min}} \approx \frac{2L^2}{\pi^2}. \quad (16)$$

For $L = 150$ this is $\kappa \approx 4.6 \times 10^3$ for a homogeneous material; the real lattice with heterogeneous (partly reduced, partly soft) elements is worse.

Gradient-type methods (steepest descent, CG, L-BFGS with a diagonal H_0) need a number of iterations that grows like $\sqrt{\kappa} \propto L$ to damp the smoothest, longest-wavelength mode. Physically: information travels one element per iteration, and a global shear has to propagate across the whole box. That is exactly what we see in practice:

	50 × 50	150 × 150
plain L-BFGS, iterations per elastic step	317	1254

A factor 3 in L gives roughly a factor 4 in iterations. The cure is to give L-BFGS a starting inverse Hessian H_0 that already knows about the long-wavelength modes.

4.3 What ALGLIB offers, and why it is not enough

`minlbfgs` in ALGLIB 3.19 has four modes for H_0 (`minlbfgssetprec*`):

mode	H_0	verdict for us
default	$\gamma_k I$	what we used so far
diagonal (<code>setprecdiag</code>)	D^{-1} , $D \approx \text{diag } \nabla^2 E$	almost useless (see below)
dense Cholesky (<code>setpreccholesky</code>)	$(U^\top U)^{-1}$ with a dense $N \times N$ factor	impossible: 45 000 ² doubles = 16 GB, $O(N^2)$ work per iteration
scale (<code>setprecscale</code>)	$\text{diag}(s_i^2)$	useless: all our variables have the same scale

Why the diagonal does not help. With periodic boundary conditions there are no boundary nodes, and in a homogeneous state every node sees exactly the same patch of elements. Hence $\text{diag } K$ takes only two values, one for the u -components and one for the v -components. A diagonal preconditioner can therefore do nothing but rescale x against y ; it has no idea about wavelengths. Worse, looking at the ALGLIB source (`prectype==2` in the two-loop recursion), the diagonal mode *replaces* the adaptive γ_k scaling by the fixed D^{-1} . Measured on elastic steps at the same gradient tolerance, diagonal preconditioning gives $1.3\times$, i.e. nothing beyond what tightening the tolerance does anyway.

A bug in the old helper. The existing helper `computeDiagonalPreconditioner` (in `FEMHessianAssembler`) returned $1/\sqrt{H_{ii}}$. ALGLIB wants the diagonal of the Hessian itself ($H_0 = D^{-1}$ is computed internally), so this would have produced $H_0 = \text{diag } \sqrt{H_{ii}}$: big steps in stiff directions. It is now fixed to return $\max(H_{ii}, \text{floor})$, but as argued above it would not have helped much anyway.

So ALGLIB has no *sparse* preconditioner. We could write our own L-BFGS, but there is a much cheaper trick.

5 The key idea: preconditioning by a change of variables

5.1 The transformation

Let P be any symmetric positive definite matrix that approximates the Hessian, and let

$$P = R R^\top \tag{17}$$

be a factorization (Cholesky). Instead of minimizing $E(\mathbf{x})$, minimize

$$\tilde{E}(\mathbf{z}) = E(\mathbf{x}_0 + R^{-\top} \mathbf{z}), \quad \mathbf{z} \in \mathbb{R}^{2n}, \tag{18}$$

starting from $\mathbf{z} = 0$, i.e. from $\mathbf{x} = \mathbf{x}_0$. By the chain rule,

$$\nabla_{\mathbf{z}} \tilde{E} = R^{-1} \nabla_{\mathbf{x}} E, \tag{19}$$

$$\nabla_{\mathbf{z}}^2 \tilde{E} = R^{-1} \nabla_{\mathbf{x}}^2 E R^{-\top}. \tag{20}$$

If P is a good approximation of $\nabla^2 E$, then $\nabla_{\mathbf{z}}^2 \tilde{E} \approx R^{-1} P R^{-\top} = I$: in the $\mathbf{z}$ variables *all directions look equally stiff*. The long-wavelength modes that were 10^3 – 10^4 times softer than the short ones are now of order one. The condition number collapses from $O(L^2)$ to $O(1)$, and L-BFGS on $\tilde{E}$ converges in a handful of iterations.

The beauty is that ALGLIB does not need to know anything about this: it just minimizes a function of $\mathbf{z}$ whose value and gradient we provide in the callback.

5.2 This is exactly preconditioned L-BFGS

It is worth convincing ourselves that this is not “some other method” but precisely L-BFGS with $H_0 = \gamma_k P^{-1}$ in the original variables.

Proposition 1. *Run L-BFGS on $\tilde{E}(\mathbf{z})$ with $\tilde{H}_0 = \gamma I$. Map every iterate back with $\mathbf{x} = \mathbf{x}_0 + R^{-\top} \mathbf{z}$. The resulting sequence $\mathbf{x}_k$ is identical to that of L-BFGS on $E(\mathbf{x})$ with $H_0 = \gamma P^{-1}$, including the line search.*

Proof. Differences transform as $\tilde{\mathbf{s}} = R^\top \mathbf{s}$ and, by (19), $\tilde{\mathbf{y}} = R^{-1} \mathbf{y}$. Hence $\tilde{\mathbf{y}}^\top \tilde{\mathbf{s}} = \mathbf{y}^\top R^{-\top} R^\top \mathbf{s} = \mathbf{y}^\top \mathbf{s}$, so all ρ_i are the same. The BFGS inverse update in $\mathbf{z}$ -space reads

$$\tilde{H}_+ = (I - \rho \tilde{\mathbf{s}} \tilde{\mathbf{y}}^\top) \tilde{H} (I - \rho \tilde{\mathbf{y}} \tilde{\mathbf{s}}^\top) + \rho \tilde{\mathbf{s}} \tilde{\mathbf{s}}^\top.$$

Assume $\tilde{H} = R^\top H R$. Using $I - \rho \tilde{s} \tilde{y}^\top = R^\top (I - \rho s y^\top) R^{-\top}$ and $I - \rho \tilde{y} \tilde{s}^\top = R^{-1} (I - \rho y s^\top) R$,

$$\tilde{H}_+ = R^\top \left[(I - \rho s y^\top) H (I - \rho y s^\top) + \rho s s^\top \right] R = R^\top H_+ R.$$

By induction $\tilde{H}_k = R^\top H_k R$ for all k , provided it holds for $k = 0$: $\tilde{H}_0 = \gamma I = R^\top (\gamma R^{-\top} R^{-1}) R = R^\top (\gamma P^{-1}) R$. The search directions match: $\tilde{d} = -\tilde{H} \tilde{g} = -R^\top H R R^{-1} g = R^\top d$, i.e. $R^{-\top} \tilde{d} = d$, so the trial points along the line are the same in $\mathbf{x}$ -space. The function values are the same by construction, and the directional derivatives too: $\tilde{g}^\top \tilde{d} = g^\top R^{-\top} R^\top d = g^\top d$. So every test of the line search gives the same answer. $\square$

Two small remarks. (i) ALGLIB still computes its adaptive scale γ_k , which in $\mathbf{z}$ -space equals $s^\top y / (y^\top P^{-1} y)$. This corrects the overall scale of P , so we do not need to worry about normalizing P . (ii) The *stopping criteria* of ALGLIB are evaluated in $\mathbf{z}$ -space: $\|\tilde{g}\| = \sqrt{g^\top P^{-1} g}$ and $\|\tilde{s}\| = \sqrt{s^\top P s}$, which are energy norms rather than Euclidean norms. That is why we use our own stopping test in $\mathbf{x}$ -space (Section 9.2).

This idea is not new in atomistic simulation: Packwood, Kermode, Mones, Bernstein, Woolley, Gould, Ortner and Csányi, *J. Chem. Phys.* **144**, 164109 (2016), used exactly such preconditioners (sparse, Laplacian-like, or built from the Hessian) and showed that the iteration count becomes essentially independent of system size.

5.3 Which P ?

The best P is the Hessian itself at the current configuration, $K = \nabla^2 E(\mathbf{x}_0)$. Near a stable minimum and for a small load increment, the energy is nearly quadratic with Hessian K , so L-BFGS with $H_0 = K^{-1}$ is essentially Newton’s method (its very first step is the Newton step) and converges in very few iterations. We compute K exactly and analytically (Section 6), with two modifications to make it a valid preconditioner (Section 7). A cheaper alternative, the reference Laplacian, is discussed in Section 9.5.

6 The analytic stiffness matrix

6.1 Second derivatives of φ with respect to the metric

The energy file already contains, generated by computer algebra, the six independent second partials of $\varphi(c_{11}, c_{22}, c_{12})$:

$$\varphi_{,1111}, \varphi_{,2222}, \varphi_{,1122}, \varphi_{,1112}, \varphi_{,2212}, \varphi_{,1212}$$

(`c1111`, ..., `c1212` in `create_d2phi_func`). As for the first derivative, the tensor second derivative $H_{ab,cd} = \partial D_{ab} / \partial C_{cd}$ is obtained by differentiating (7) once more with the same convention. For example $dD_{11} = \varphi_{,1111} dc_{11} + \varphi_{,1122} dc_{22} + \varphi_{,1112} dc_{12}$, and dc_{12} is shared between dC_{12} and dC_{21} , so $H_{11,12} = H_{11,21} = \frac{1}{2} \varphi_{,1112}$. Likewise $D_{12} = \frac{1}{2} \varphi_{,12}$ gives $H_{12,12} = H_{12,21} = \frac{1}{4} \varphi_{,1212}$. With the ordering (11, 12, 21, 22) of the index pairs:

$$H = \begin{pmatrix} \varphi_{,1111} & \frac{1}{2} \varphi_{,1112} & \frac{1}{2} \varphi_{,1112} & \varphi_{,1122} \\ \frac{1}{2} \varphi_{,1112} & \frac{1}{4} \varphi_{,1212} & \frac{1}{4} \varphi_{,1212} & \frac{1}{2} \varphi_{,2212} \\ \frac{1}{2} \varphi_{,1112} & \frac{1}{4} \varphi_{,1212} & \frac{1}{4} \varphi_{,1212} & \frac{1}{2} \varphi_{,2212} \\ \varphi_{,1122} & \frac{1}{2} \varphi_{,2212} & \frac{1}{2} \varphi_{,2212} & \varphi_{,2222} \end{pmatrix}. \quad (21)$$

This is *exactly* the rule implemented in your `AcousticTensor::computeHessian` (“factor 0.5 when one index pair is off-diagonal, 0.25 when both are”), so the physics is the same as in your existing code. I verified numerically that the coded `ddphi_func` components equal the finite-difference derivatives of `dphi_func` (ratio 1.000000 for all six). (Finite differences were only used as a *check*; the tangent itself is fully analytic.)

6.2 The Lagrangian tangent

We need $\mathbb{A}_{iK,jL} = \partial^2 \varphi / \partial F_{iK} \partial F_{jL} = \partial P_{iK} / \partial F_{jL}$. Differentiate $P_{iK} = D_{ab} B_{ab,iK}$:

$$\mathbb{A}_{iK,jL} = B_{ab,iK} H_{ab,cd} B_{cd,jL} + D_{ab} \frac{\partial^2 (C_{\text{red}})_{ab}}{\partial F_{iK} \partial F_{jL}}. \quad (22)$$

From $(C_{\text{red}})_{ab} = G_{ma} G_{mb}$ and $\partial G_{ma} / \partial F_{iK} = \delta_{mi} Z_{Ka}$,

$$\begin{aligned} \frac{\partial^2 (C_{\text{red}})_{ab}}{\partial F_{iK} \partial F_{jL}} &= \delta_{ij} (Z_{Ka} Z_{Lb} + Z_{Kb} Z_{La}), \\ D_{ab} \delta_{ij} (Z_{Ka} Z_{Lb} + Z_{Kb} Z_{La}) &= \delta_{ij} 2(Z D Z^T)_{KL} = \delta_{ij} S_{KL}. \end{aligned}$$

So the tangent is the sum of a *material* part and a *geometric* (initial-stress) part:

$$\mathbb{A} = B^T H B + I \otimes S, \quad B_{ab,iK} = Z_{Ka} G_{ib} + Z_{Kb} G_{ia}, \quad G = F Z, \quad S = 2 Z D Z^T. \quad (23)$$

Here $\mathbb{A}$ is stored as a 4×4 matrix with row index $2i + K$ and column index $2j + L$, B as a 4×4 matrix with row $2a + b$ and column $2i + K$, and H as the 4×4 matrix (21) divided by a^2 .

In your `AcousticTensor` this same object is built with ITensor contractions: the tensor `T53` is $\partial C_{mn} / \partial F_{ij} = \delta_{mj} F_{in} + \delta_{nj} F_{im}$ (contracted with Z it becomes our B), `T54` produces the $\delta_{ij} S_{KL}$ term, `mm1...mm4` are the Z matrices, and `f1+f2` is the sum in (23). The new code computes (23) with fixed-size Eigen matrices:

```
Eigen::Matrix4d element_lagrangian_tangent(const Eigen::Matrix2d &F,
    const Eigen::Matrix2d &Z, const Eigen::Matrix2d &dE_dC_reduced,
    const HessianComponents &h, double normalisation) {
    // H: tensor second derivative, same 1/2, 1/4 factors as AcousticTensor::computeHessian
    const double d1112 = 0.5 * h.c11_c12, d2212 = 0.5 * h.c22_c12, d1212 = 0.25 * h.c12_c12;
    Eigen::Matrix4d H;
    H << h.c11_c11, d1112, d1112, h.c11_c22,
        d1112, d1212, d1212, d2212,
        d1112, d1212, d1212, d2212,
        h.c11_c22, d2212, d2212, h.c22_c22;
    H /= normalisation;
    // B(2a+b, 2i+K) = d(C_red)_ab / dF_iK = Z_Ka G_ib + Z_Kb G_ia, G = F Z
    const Eigen::Matrix2d G = F * Z;
    Eigen::Matrix4d B;
    for (int a = 0; a < 2; a++) for (int b = 0; b < 2; b++)
        for (int i = 0; i < 2; i++) for (int K = 0; K < 2; K++)
            B(2*a+b, 2*i+K) = Z(K,a)*G(i,b) + Z(K,b)*G(i,a);
    Eigen::Matrix4d A = B.transpose() * H * B;
    // geometric term delta_ij S_KL, S = 2 Z (dE/dC) Z^T
    const Eigen::Matrix2d S = 2.0 * Z * dE_dC_reduced * Z.transpose();
    for (int i = 0; i < 2; i++) for (int K = 0; K < 2; K++) for (int L = 0; L < 2; L++)
        A(2*i+K, 2*i+L) += S(K,L);
    return A;
}
```

6.3 From the tangent to the element stiffness

Using $\partial F_{iK} / \partial x_{a,j} = \delta_{ij} \partial N_a / \partial X_K$ twice,

$$K_{(a,i),(b,j)}^e = \frac{\partial^2 E_e}{\partial x_{a,i} \partial x_{b,j}} = A_e \sum_{K,L} \mathbb{A}_{iK,jL} \frac{\partial N_a}{\partial X_K} \frac{\partial N_b}{\partial X_L}. \quad (24)$$

In matrix form, with the 4×6 matrix $\mathcal{G}_{2i+K, 2a+j} = \delta_{ij} \partial N_a / \partial X_K$,

$$K^e = A_e \mathcal{G}^T \mathbb{A} \mathcal{G} \quad (6 \times 6). \quad (25)$$

This is the same formula as in `FEMHessianAssembler::computeElementStiffness`.

6.4 Getting the second derivatives without ITensor

The existing `BaseLatticeCalculator::calculate_dseconderivative` returns an `ITensor`. Building and reading `ITensors` per element is what made the old assembler slow (7.2s serially for 5000 elements). So I added one virtual method,

```
virtual HessianComponents calculate_dseconderivative_components(
    const Eigen::Matrix2d& C, const std::function<double(double)>& dpot,
    const std::function<double(double)>& d2pot);
```

which returns the six raw partials in the `HessianComponents` struct that already existed in the base header. The base-class default reads them back from the `ITensor` (so any future calculator works automatically); `Strain_Energy_LatticeCalculator`, `SquareLatticeCalculator` and `TriangularLatticeCalculator` override it and never touch `ITensor`. For the strain energy, the huge generated expressions were moved into `create_d2phi_components_func`; `create_d2phi_func` now calls it and packs the `ITensor`, so the expressions exist only once.

6.5 Global assembly

K is the $2n \times 2n$ sparse matrix $\sum_e (\text{scatter of } K^e)$. The DOF of local entry (a, i) is s_a for $i = 0$ and $s_a + n$ for $i = 1$, with $s_a = \text{full_mapping}[\text{nn}[a]].\text{second}$. Couplings across the periodic boundary come for free because images carry the DOF of their original node.

The implementation (`FastStiffnessAssembler::assembleStiffness`) is organized for speed:

1. **Pattern once.** The sparsity pattern depends only on the connectivity. It is built once (with `setFromTriplets`) and, for every element, the positions of its 36 entries in the compressed value array are stored (`k_slots_`). A hash of the connectivity (`mesh_signature`) detects when remeshing changes the mesh, and only then is the pattern rebuilt.
2. **Values in parallel.** An OpenMP loop over elements computes F , reduces C , evaluates D and H , forms $\mathbb{A}$ and K^e , and adds the 36 numbers directly into the value array with `#pragma omp atomic`.

At 50×50 : 2.9 ms the first time (pattern included), 0.4 ms afterwards, versus 7.2 s for the serial `ITensor` loop. At 150×150 assembly takes about 8 ms.

6.6 Validation of K

The benchmark executable (`benchmark_preconditioner --validate`) checks, on a real relaxed configuration with 3106 of 5000 elements Lagrange-reduced:

- element by element against the `ITensor computeElementStiffness`: $\max |K_{\text{fast}}^e - K_{\text{ITensor}}^e| / \max |K^e| = 3 \times 10^{-16}$;
- symmetry: $\max |K - K^T| = 10^{-14}$;
- K against the analytic gradient: $\|K\mathbf{v} - [\mathbf{g}(\mathbf{x} + \epsilon\mathbf{v}) - \mathbf{g}(\mathbf{x} - \epsilon\mathbf{v})]/2\epsilon\| / \|K\mathbf{v}\| \approx 10^{-7}$ for random unit $\mathbf{v}$ (the finite-difference noise level);
- gradient against the energy, at a non-equilibrium point: $[E(\mathbf{x} + \epsilon\mathbf{v}) - E(\mathbf{x} - \epsilon\mathbf{v})]/(2\epsilon \mathbf{g} \cdot \mathbf{v}) = 1.0000$.

In other words, K is the exact Hessian of the discrete energy (4), the gradient is its exact first derivative, and both coincide with your existing `ITensor` implementation.

7 Turning K into a preconditioner

Two things prevent us from Cholesky-factorizing the exact Hessian directly.

7.1 Indefiniteness: element-wise PSD projection

Near instabilities, and in the strongly non-equilibrium states after noise or during an avalanche, some elements are in a non-convex part of φ and $\mathbb{A}$ has negative eigenvalues. Then K is indefinite, Cholesky fails, and even if it did not, $H_0 = K^{-1}$ would not be positive definite and the L-BFGS direction might point uphill.

The fix is a standard trick from projected Newton methods: project every *element* tangent onto the positive semi-definite cone before assembly. $\mathbb{A}$ is symmetric (major symmetry of a second derivative), so

$$\mathbb{A} = Q\Lambda Q^\top \longrightarrow \mathbb{A}^+ = Q \max(\Lambda, 0) Q^\top. \quad (26)$$

Then $K^{e+} = A_e \mathcal{G}^\top \mathbb{A}^+ \mathcal{G}$ is PSD, and a sum of PSD matrices is PSD, so the assembled K^+ is PSD. Where the material is stable nothing changes; where an element is unstable its unstable direction simply gets zero stiffness in the preconditioner. That is actually a sensible thing to tell the optimizer: “you may take large steps in this direction”. The line search guarantees descent anyway.

The 4×4 symmetric eigen-decomposition per element costs almost nothing (`Eigen::SelfAdjointEigenSolver<Matrix4d>`).

7.2 Rigid translations: a small shift

With periodic boundary conditions a rigid translation of all nodes does not change any F , hence $K\mathbf{t} = 0$ for the two translation vectors $\mathbf{t} = (1, \dots, 1, 0, \dots, 0)$ and $(0, \dots, 0, 1, \dots, 1)$: K has a two-dimensional null space. We factorize

$$P = K^+ + \sigma I, \quad \sigma = 10^{-6} \overline{\text{diag } K^+}. \quad (27)$$

The shift is far below the smallest non-zero eigenvalue ($\sim (2\pi/L)^2$ times the typical stiffness, i.e. $\sim 10^{-3}$ relative at $L = 150$), so it does not spoil the preconditioner. It is harmless for the translation directions: the net force $\sum_a \partial E / \partial \mathbf{x}_a$ vanishes identically (because $\sum_a \nabla N_a = 0$ on every element), so the gradient has no component along the null space. If a factorization nevertheless fails, the code multiplies σ by 100 and retries (up to 6 times), and as a last resort falls back to plain L-BFGS.

8 Sparse Cholesky and the two half-solves

8.1 Factorization

Sparse Cholesky first chooses a fill-reducing permutation Π (AMD ordering, computed once per mesh in the “analyze” phase) and then computes a lower-triangular L with

$$\Pi P \Pi^\top = LL^\top. \quad (28)$$

Hence $P = RR^\top$ with

$$R = \Pi^\top L, \quad R^{-1} = L^{-1} \Pi, \quad R^{-\top} = \Pi^\top L^{-\top} \quad (29)$$

(a permutation matrix is orthogonal, $\Pi^{-1} = \Pi^\top$).

Two back-ends are implemented behind a small interface (`SparseCholeskyBackend`):

- **CHOLMOD** (SuiteSparse, supernodal). Columns of L with identical sparsity are grouped into dense “supernodes” that are processed with BLAS-3, which is fast and can be multi-threaded. Used when CMake / Makefile.cluster finds SuiteSparse (`-DUSE_CHOLMOD=1`).
- **Eigen SimplicialLLT** (column by column, serial). Always available; the fallback.

Both use the shift directly in the numeric factorization (`cholmod_factorize_p` with $\beta = \sigma$, resp. `SimplicialLLT::setShift`), so K itself is never modified.

Measured at 150×150 (45 000 unknowns): factorization 0.032 s with CHOLMOD, 0.33 s with Eigen.

8.2 Applying R^{-1} and R^{-T}

We never form R^{-1} . The two operations needed per energy evaluation are

$$\text{gradient to } \mathbf{z}: \quad \tilde{\mathbf{g}} = R^{-1} \mathbf{g} = L^{-1}(\Pi \mathbf{g}) \quad (\text{permute, then forward substitution}) \quad (30)$$

$$\text{step back to } \mathbf{x}: \quad \mathbf{x} - \mathbf{x}_0 = R^{-T} \mathbf{z} = \Pi^T(L^{-T} \mathbf{z}) \quad (\text{back substitution, then inverse permutation}) \quad (31)$$

With CHOLMOD these are `cholmod_solve2` calls with `CHOLMOD_P`, `CHOLMOD_L` and `CHOLMOD_Lt`, `CHOLMOD_Pt`; with Eigen, `permutationP()`, `matrixL().solveInPlace()` and `matrixU().solveInPlace()`, `permutationPinv()`. Each triangular solve costs $O(\text{nnz}(L))$; together about 4 ms at 150×150 , versus about 2 ms for one energy+gradient evaluation.

9 The algorithm as implemented

9.1 One call of `PreconditionedLBFGS::optimize`

For every relaxation (one load step, or one relaxation inside the remeshing loop):

1. **Build the preconditioner** at the starting point $\mathbf{x}_0$ (`buildPreconditioner`): assemble $K^+(\mathbf{x}_0)$ (≈ 8 ms), factorize $K^+ + \sigma I$ (≈ 32 ms). The symbolic analysis (ordering) is redone only when the mesh changes.
2. **Run ALGLIB on $\mathbf{z}$** starting from $\mathbf{z} = 0$, with the callback `transformed_energy`:

```
static void transformed_energy(const alglib::real_1d_array &z, double &func,
                             alglib::real_1d_array &grad, void *ptr) {
    auto *ctx = static_cast<TransformContext *>(ptr);
    double *x = ctx->x_work.getcontent();
    factor->apply_inverse_transpose(z.getcontent(), x, n); // x - x0 = R^{-T} z
    for (int i = 0; i < n; i++) x[i] += ctx->x0[i];
    minimize_energy_with_triangles(ctx->x_work, func, ctx->gx_work, ctx->userData);
    factor->apply_inverse(ctx->gx_work.getcontent(), grad.getcontent(), n); // R^{-1} g
    // stop on the true force, measured in x-space (see below)
    if (max_i |g_i| < grad_tol) { remember x; alglib::minlbfgsrequesttermination(*ctx->state)
        ; }
}
```

Your energy routine `minimize_energy_with_triangles` is called unchanged.

3. **Map back:** $\mathbf{x} = \mathbf{x}_0 + R^{-T} \mathbf{z}_{\text{final}}$ and return it in the same array the old optimizer used, so all the code after the minimization (remeshing decision, stresses, output) is untouched.

K is *not* updated during the ALGLIB iterations. What adapts is the L-BFGS memory (the 13 pairs), which corrects $H_0 = K^{-1}$ with the curvature actually encountered. K cannot be swapped in the middle of a run: a new R would define new variables $\mathbf{z}$ and make the stored pairs meaningless. Changing R therefore always means restarting ALGLIB, which is what the restart logic below does.

9.2 Stopping criterion

With the change of variables, ALGLIB’s own criteria would be measured in $\mathbf{z}$ -space (energy norms, see above), and its automatic criterion (a small step, $\|\tilde{\mathbf{s}}\| \leq 10^{-6}$) would mean something different from the production runs. Instead we stop when the true force is small,

$$\max_i \left| \frac{\partial E}{\partial x_i} \right| < \text{grad_tol} = 10^{-6}, \quad (32)$$

checked inside the callback; ALGLIB is then asked to terminate (`minlbfgsrequesttermination`) and the point that met the criterion is returned. The same test is used for all preconditioners, so comparisons are fair. (For reference, the plain production runs stop at $\max |g|$ between 10^{-8} and 5×10^{-5} ; energies of the two solvers agree to 10^{-10} .)

9.3 Restarts when the preconditioner goes stale

During a large rearrangement (an avalanche, or relaxing the initial noise) the configuration moves far away from $\mathbf{x}_0$ and $K(\mathbf{x}_0)$ becomes a poor model. ALGLIB may then stop early for numerical reasons without reaching `grad_tol`. In that case `optimize` rebuilds K at the current point, refactorizes, and starts a new ALGLIB run from there (at most 3 restarts). In the tests below no restart was ever needed on avalanche steps; it is a safety net.

9.4 Refresh policy

By default (`refresh_every = 1`) K is rebuilt at the start of every load step. One can reuse the factorization for several steps (`--precond-refresh=N`), optionally refreshing early when a solve needed more than a given number of iterations. With CHOLMOD the setup is so cheap (40 ms) that this buys nothing measurable ($14.9\times$ vs $15.2\times$), so the default is to refresh every step.

9.5 The Laplacian variant

A much simpler preconditioner uses only the mesh: the scalar P1 Laplacian of the reference configuration,

$$\mathcal{L}_{s_a s_b} = \sum_e A_e \nabla N_a \cdot \nabla N_b, \quad P = \begin{pmatrix} \mathcal{L} + \sigma I & 0 \\ 0 & \mathcal{L} + \sigma I \end{pmatrix}, \quad (33)$$

which does not depend on $\mathbf{x}$ at all, is factorized once per mesh, and acts on u and v separately with the same $n \times n$ factor. It captures the long-wavelength problem (the L^2 condition number) but knows nothing about the actual elastic constants, anisotropy or the reduced elements. It gives about $3\times$; the true stiffness gives $15\times$.

9.6 Cost of one elastic step at 150×150

	plain L-BFGS	stiffness preconditioner
assembly of K	–	0.008 s
factorization (CHOLMOD)	–	0.032 s
iterations	~ 1250	~ 37
function evaluations	~ 2490	~ 53
cost per evaluation	≈ 2.2 ms	≈ 6.2 ms
total per step	5.5 s	0.37 s

Each evaluation is about three times more expensive (two triangular solves on top of the energy), but we need about 40 times fewer of them.

10 Results

All benchmarks were run with `benchmark_preconditioner`. At each load step all methods start from the *same* configuration, and the state is then advanced with the solution of the first method (plain production L-BFGS), so every method solves exactly the same sequence of problems. The loading is the production Zanzotto shear, PBC, fixed mesh, M1 Pro with 10 threads.

10.1 Elastic steps

method	50 × 50		150 × 150 (CHOLMOD)	
	its/step	speedup	its/step	speedup
plain (production)	245	1.0	1254	1.0
plain, same tolerance	228	1.1	—	—
diagonal (ALGLIB)	237*	1.3*	—	—
Laplacian	64	3.2	207 [†]	3.0 [†]
stiffness	10.7	12.3	37	14.9

* measured before the gradient fix; [†] with the Eigen back-end run. With Eigen instead of CHOLMOD the stiffness preconditioner gives 7.8× at 150 × 150, the difference being entirely the factorization time.

In all cases the preconditioned solutions coincide with the production solutions to about 10^{-10} in energy. Note that the iteration count of the preconditioned solver still grows somewhat with size (11 → 37): $K(\mathbf{x}_0)$ is not the Hessian at the solution, the problem is nonlinear, and the absolute force tolerance involves more degrees of freedom. But the gap to plain L-BFGS widens with size, which is what matters for large systems.

10.2 Avalanches

To see many instabilities quickly, a 50 × 50 run with ten times larger increments ($\Delta\alpha = 6 \times 10^{-4}$, 150 steps, α from 0.14 to 0.23) was used. It contains 9 energy drops. Total time excluding step 0:

method	elastic steps (140)	avalanche steps (9)		total
plain	43.5 s	5.8 s		49.2 s
Laplacian	15.2 s	2.1 s		17.4 s
stiffness	2.8 s	1.1 s	3.9 s (12.6×)	

On the avalanche steps themselves the stiffness solver needed 50–153 iterations against 650–1180 for plain L-BFGS, with no restarts. In 7 of the 9 avalanches it ended in exactly the same state as plain L-BFGS ($|\Delta E| \sim 10^{-11}$). In two it did not: once it found a state lower by 0.076 (the Laplacian found the same one), once a state higher by 0.047. This is discussed in Section 11.

10.3 The production binary

Running the real program for 45 s at 50 × 50:

configuration	load steps in 45 s
plain, before the gradient fix	182
plain, after the gradient fix	212
<code>--precond=stiffness --precond-from-step=1</code>	2062

With `--precond-from-step=1` the first 182 steps match the plain run to 1.2×10^{-10} in energy and 1.2×10^{-7} in stress.

A full 150×150 production-style run (positive shear, seed 42, remeshing on) was started with the stiffness preconditioner: after 50 minutes it had completed 6586 steps, with elastic steps at 0.30 s against 2.87 s in the old run ($9.6\times$, including remeshing overhead), on track for ~ 1.8 h instead of 11.7 h.

11 Physics caveats: path dependence

A preconditioner changes the *path* of the minimizer, not the energy landscape. On a convex problem the path is irrelevant: there is one minimum and every method finds it. That is why elastic steps agree to 10^{-10} . But whenever the starting point is far from equilibrium in a non-convex landscape, different paths may end in different local minima. Neither answer is “wrong”: quasi-static minimization does not define which basin the system falls into after an instability; that is decided by the (fictitious) minimization dynamics.

This matters in two places.

Step 0. The first load step relaxes a lattice sheared to $\alpha = 0.14$ with 4% random noise on the positions: a violent, strongly non-convex relaxation (~ 5000 iterations at 150×150). At 50×50 , plain L-BFGS ends at $E = 14.09$ with shear stress $+7.1 \times 10^{-3}$, while the preconditioned solver ends at $E = 7.99$ with stress -4.1×10^{-3} : a different initial microstructure, i.e. effectively a different sample. The option `--precond-from-step=N` runs the plain solver (including its remeshing) for load steps $< N$. With $N = 1$ (the default in the job scripts) the initial state is identical to that of a plain run, at a cost of about 20 s out of hours.

(A side remark: at 150×150 the step-0 energy of today’s code, with or without preconditioner, is 122.805, while the old run from 9–10 September gave 116.062. The step-0 relaxation is so sensitive that the code changes made since then, this morning’s optimization of `minimize_energy_with_triangles` and `lagrange::reduce` and/or the gradient fix, send it to a different minimum. New runs should therefore be compared with new plain runs, or statistically.)

Avalanches. As seen above, most avalanches end in the same state, but not all. The scientific output of these simulations is statistical (avalanche size distributions, stress drops, dislocation statistics), so the right validation is statistical: run a few seeds with and without preconditioner and compare the distributions.

12 Stability analysis with the same machinery: the lowest eigenvalues of K

12.1 What we want and how it was done before

The *unprojected* stiffness K is the Hessian of the discrete energy. At a relaxed configuration all its eigenvalues are ≥ 0 ; an avalanche starts when the lowest nontrivial eigenvalue $\lambda_{\min}$ crosses zero (a saddle-node bifurcation), and the corresponding eigenvector is the soft mode that becomes unstable. The analysis code (`analyze_data_from_folder` in `src/experiments/data_analysis.cpp`) therefore loads saved configurations and computes the 100 lowest eigenpairs of K :

1. K is assembled by `FEMHessianAssembler::assembleGlobalStiffness`, element by element through `ITensor`, serially;

2. `computeSmallestEigenvaluesIterative_spectra` runs Spectra's shift-invert Lanczos with `SparseSymShiftSolve`, which factorizes $K - \sigma I$ with a sparse LU (`Eigen::SparseLU`), with $\sigma = -0.01 \max_i |K_{ii}|$;
3. the two rigid translations (eigenvalue 0 under periodic boundary conditions) come out among the 100 and are recognized afterwards (`detectRigidBodyModes`).

At 150×150 this takes 20 s per configuration: 3.7 s for the ITensor assembly and 16 s for the eigensolver (with the release ITensor library `libitensor.a`; with the debug library `libitensor-g.a`, which `CMakeLists.txt` used to prefer and now links only with `-DITENSOR_DEBUG_LIB=ON`, the assembly alone takes 65 s).

12.2 Shift-invert Lanczos in one paragraph

Lanczos finds the *extreme* eigenvalues of a symmetric operator quickly, but the lowest eigenvalues of a stiffness matrix are extreme only in a useless sense: they are tiny and clustered compared with the spread $\lambda_{\max} \sim 8\bar{K}_{ii}$. The spectral transformation [12] applies Lanczos to $(K - \sigma I)^{-1}$ instead, whose eigenvalues are

$$\nu_k = \frac{1}{\lambda_k - \sigma}, \quad \lambda_k = \sigma + \frac{1}{\nu_k}. \quad (34)$$

The eigenvalues closest to σ become the largest ν 's and are well separated from the rest, so they converge in a few tens of iterations. Every Lanczos iteration costs one solve with $K - \sigma I$, so the method lives or dies with the factorization of $K - \sigma I$.

12.3 Three improvements

1. Fast assembly. K is assembled by the `FastStiffnessAssembler` of Section 6 with `project_psd=false`: exactly the same matrix (checked: $\max |K_{\text{fast}} - K_{\text{ITensor}}| = 7 \times 10^{-13}$), in 25 ms instead of 3.7 s.

2. Cholesky instead of LU: put the shift *below* the spectrum. If $\sigma < \lambda_{\min}(K)$, then $K - \sigma I$ is symmetric positive definite and we can use the sparse Cholesky factorization of Section 8 (`CHOLMOD`, 45 ms at 150×150), which is several times cheaper than LU in time and memory and gives faster solves. We do not know $\lambda_{\min}$ in advance, but we do not need to: by Sylvester's law of inertia [13], the number of negative pivots of a symmetric factorization of $K - \sigma I$ equals the number of eigenvalues of K below σ . In particular

$$\text{Cholesky of } K - \sigma I \text{ succeeds} \iff \lambda_{\min}(K) > \sigma. \quad (35)$$

So we try $\sigma_0 = -10^{-4} \bar{K}_{ii}$ and, while the factorization fails, set $\sigma \leftarrow 10\sigma$. A failure is not wasted: it is a rigorous statement that K has an eigenvalue below σ , i.e. *the configuration is unstable*. After m failures, $\lambda_{\min} \in (\sigma, \sigma/10]$, so the shift is always within a factor 10 of $|\lambda_{\min}|$ and the lowest modes converge fast.

3. Deflate the translations instead of computing them. Under PBC every node is free, and a uniform translation $\mathbf{t}_u = (1, \dots, 1, 0, \dots, 0)$ or $\mathbf{t}_v = (0, \dots, 0, 1, \dots, 1)$ does not change any deformation gradient, so $K\mathbf{t}_u = K\mathbf{t}_v = 0$ exactly. They are therefore also eigenvectors of $(K - \sigma I)^{-1}$, and with the orthogonal projector $Q = I - \hat{\mathbf{t}}_u \hat{\mathbf{t}}_u^\top - \hat{\mathbf{t}}_v \hat{\mathbf{t}}_v^\top$ (which just subtracts the mean of the u block and of the v block) the operator

$$\mathcal{A}\mathbf{x} = Q(K - \sigma I)^{-1}Q\mathbf{x} \quad (36)$$

is symmetric, has the same eigenpairs as $(K - \sigma I)^{-1}$ on the complement of the translations, and maps the translations to 0, where Lanczos never looks. No eigenpairs are spent on them and no detection step is needed. There is a second benefit: the translation eigenvalue 0 is double, and a single-vector Lanczos can return only one copy of a double eigenvalue. With the old code and 3 requested values this happens: one translation, λ_1 and λ_2 come back, so the number of “rigid modes” found depends on how many values are requested. The code checks $\|Kt\|_\infty \leq 10^{-9} \max |K_{ij}|$ before deflating, so with fixed boundaries (no translation modes) nothing is projected.

12.4 Results

Same relaxed 150×150 configuration ($n = 45\,000$; 3 load steps of the benchmark), M1 Pro, CHOLMOD, release ITensor library for the old path:

values requested (nontrivial modes)	100 (98)	12 (10)	3 (1: $\lambda_{\min}$)
old: ITensor assembly	3.7 s	3.7 s	3.6 s
old: Spectra + SparseLU	16.2 s	6.9 s	5.2 s
old total	19.9 s	10.5 s	8.8 s
new: assembly	0.03 s	0.02 s	0.02 s
new: Cholesky	0.05 s	0.04 s	0.04 s
new: Lanczos (solves)	2.22 s (248)	0.18 s (45)	0.08 s (21)
new total	2.32 s	0.25 s	0.15 s
speedup	8.6×	42×	59×

The eigenvalues agree with the old ones to a relative 1.5×10^{-11} (98 modes), and the residuals $\|Kv - \lambda v\|/|\lambda|$ are $\leq 1.5 \times 10^{-11}$. With the debug ITensor library the old path takes 79 s for 100 values. With $\lambda_{\min}$ at 0.15 s it becomes affordable to monitor stability *at every load step* of a simulation, not only at a few saved configurations.

In analyze_data_from_folder. On saved configurations 1, 3, 5 of the 150×150 preconditioned run (100 values, release ITensor), the eigenvalue part takes 2.2 s (fast) against 14–15 s (legacy) per configuration; the output file `eigenvalues_vs_shear.dat` is the same to 11 digits in the first nontrivial eigenvalue, with 2 rigid modes detected in both cases.

Unstable test. To exercise the shift search, the solver was applied to $K - cI$ with c chosen so that three eigenvalues are negative. The first shift $\sigma_0 = -5.5 \times 10^{-3}$ fails (instability detected), $\sigma = -5.5 \times 10^{-2}$ succeeds, and the five lowest eigenvalues -3.389×10^{-2} , -1.122×10^{-3} , -8.776×10^{-4} , 1.018×10^{-2} , 1.891×10^{-2} are reproduced to all printed digits.

Usage. From C++:

```
#include "optimization/StiffnessSpectrum.h"

StiffnessSpectrumOptions opt;
opt.n_modes = 10;           // lowest 10 nontrivial modes
StiffnessSpectrum s = lowest_stiffness_modes(x, &userData, opt);
// s.eigenvalues (ascending), s.eigenvectors (solver layout [u..., v...]),
// s.failed_shifts > 0 <=> K has an eigenvalue below the first trial shift
```

The command `benchmark_preconditioner --eig 100` reproduces the comparison above. The function `analyze_data_from_folder` uses the fast path by default and the old one with `--eig-`

`solver=legacy` (flag of `lattice_triangulation`, passed as the last argument of the call in `main.cpp`). The fast path adds the two translations back explicitly (normalized $\hat{t}_u, \hat{t}_v$, eigenvalue = Rayleigh quotient $\approx 10^{-17}$), so the rigid-mode detection and the output columns are unchanged.

12.5 Monitoring stability during the run

The same routine runs inside the loading loop (`StabilityMonitor`, `src/experiments/stability_monitor.cpp`), always on relaxed (POST) states:

- *grid*: every N -th load step (`--eig-every=N`); at 150×150 one computation costs ≈ 0.15 s against ≈ 0.42 s per load step, so $N = 5$ adds $\approx 7\%$;
- *refine*: every step while $\lambda_{\min} < 0.2 \lambda_{\text{ref}}$, where λ_{ref} is the first value computed after the last avalanche (`--eig-refine`); near a saddle-node $\lambda_{\min} \propto \sqrt{\alpha_c - \alpha}$, so the interesting part of the curve is the last few steps;
- *before/after avalanche* (`--eig-at-avalanche=1`): the relaxed state of the previous step, `POST( $i-1$ )`, the last stable state before the avalanche, is kept in memory (positions; its mesh is the one step i starts with), and its eigenpairs are computed retroactively once step i is recognized as an avalanche; `POST( $i$ )` is computed as well.

Eigenvalues go to `eigen_log.csv`; the lowest two nontrivial modes of `POST( $i-1$ )` go to `eigen_modes/soft_modes_XXXXX.vtk`, next to the PRE-avalanche configuration with the same id. Translations are projected out of the solver and, as a safeguard, any vector with more than half of its norm in the translations would be skipped.

A first 50×50 run (`--eig-every=5 --eig-at-avalanche=1`) shows the expected behaviour, and one instructive case: at steps 1281–1287 $\lambda_{\min}$ decreases from 3.7×10^{-3} to 1.3×10^{-3} and jumps to 3.4×10^{-2} at step 1288, with a 26% stress drop and a 1.4% energy drop; since remeshing accepted no topological change there, this instability is *not* counted as an avalanche and no configuration is saved. The eigenvalue log sees it anyway.

12.6 Saddle-node analysis

Near a fold (saddle-node) instability at load α_c the lowest eigenvalue at the equilibrium behaves as $\lambda_{\min} \simeq \sqrt{s(\alpha_c - \alpha)}$, so $\lambda_{\min}^2$ is *linear* in α : plotting $\lambda_{\min}^2$ instead of $\lambda_{\min}$ turns the approach into a straight line whose zero is α_c . `plot_saddle_node.py <run_dir>` does this automatically: it finds every instability (a stress decrease larger than two typical elastic increments, saved or not), fits $\lambda_{\min}^2 = A + B\alpha$ on the largest tail of ≥ 3 points with $R^2 \geq 0.995$, and plots $\lambda_{\min}$ with the fitted square roots, the collapse $\lambda_{\min}/\sqrt{s\Delta\alpha}$ versus $(\alpha_c - \alpha)/\Delta\alpha$ on the slope $1/2$, and where α_c falls relative to the last stable step.

The square-root regime turned out to be short, a few to ten load steps, so sampling every 5 steps resolves few approaches. Two additions to the monitor fix this: the predictive refinement (`--eig-refine-ahead`: every step once the linear extrapolation of $\lambda_{\min}^2$ reaches zero within two grid intervals) and the retroactive computations (`--eig-retro=4`: the last four relaxed states are kept in memory and computed when an instability is detected). In the 50×50 test run this gives 63 fits out of 86 instabilities, 59 of them with α_c within the last load step, for 124 extra computations (2 s). Of the remaining 23, 16 approach so fast that the square-root regime is shorter than one load step, and 7 show no softening at all before the jump: $\lambda_{\min}$ stays finite, i.e. these are not folds of a smooth energy (possibly caused by the non-smoothness of the Lagrange-reduced energy or by the remeshing).

Further options. If many modes are needed repeatedly, LOBPCG [14] preconditioned with the PSD-projected Cholesky factor that the L-BFGS solver already has would avoid the shift search altogether; and since the Cholesky of K^+ is computed at every load step anyway, a single extra factorization of $K + \epsilon I$ (no projection) gives a yes/no stability test per step for 45 ms.

13 Related work: what is known and what is new

It is worth being honest about which parts of this note are textbook material.

Known ingredients.

- *L-BFGS with a nontrivial initial inverse Hessian.* L-BFGS [1, 2] only needs a matrix H_0 to start its two-loop recursion; that H_0 may be any SPD matrix is standard [3]. Section 5 (change of variables $\Leftrightarrow H_0 = \gamma P^{-1}$) is a reformulation of this well-known fact, needed only because ALGLIB [4] does not accept a sparse H_0 .
- *The stiffness matrix as H_0 in finite elements.* Quasi-Newton methods started from the factorized stiffness go back to Matthies and Strang [5] and are the “quasi-Newton” option of commercial FE codes.
- *Sparse, physics-based preconditioners for L-BFGS in atomistics.* Jiang et al. [6] precondition L-BFGS with part of the Hessian of molecular force fields; Packwood et al. [7] propose a sparse Laplacian-like preconditioner for condensed matter, show that the gain grows with system size (our $\kappa \sim L^2$ argument), and ship it in ASE; Mones et al. [8] build it from the Hessian of the force-field terms, keeping only the positive definite part, which is what our PSD projection does.
- *L-BFGS with a prefactorized Laplacian for hyperelasticity* [9] (computer graphics): essentially our Laplacian variant.
- *Element-wise projection of indefinite tangents* [10] is standard in “projected Newton” methods.
- *Shift-invert Lanczos, inertia, sparse Cholesky* [12, 13, 16, 15] are classical numerical linear algebra.

What seems specific to this work.

- The analytic tangent of a $GL(2, \mathbb{Z})$ -invariant, Lagrange-reduced energy [18, 19], including the piecewise-constant reduction matrix Z (eq. (23)), and its use as a preconditioner in a landscape where the Hessian *jumps* whenever an element changes reduction.
- The behaviour through avalanches and remeshing: a K^+ built at the start of a load step remains effective across reduction switches, with restarts when it goes stale.
- The effect of the minimizer on metastable-state selection after instabilities (Section 11), which matters for avalanche statistics.

To our knowledge the Lagrange-reduction plasticity simulations published so far use plain first-order minimizers (L-BFGS, FIRE-type dynamics [11]). A publication would need more than a speedup: a size-scaling study ($L = 50 \dots 500$), comparisons with FIRE, nonlinear CG and Newton with the same K , and statistical equivalence of avalanche distributions over many seeds; or, for a numerical-analysis journal, a genuinely new ingredient (convergence theory for the non-smooth reduced energy, instability detection or saddle search built on the factorization, reuse of the factorization across remeshing, 3D).

14 Implementation map

file / symbol	role
include/optimization/ PreconditionedLBFGS.h, src/optimization/ PreconditionedLBFGS.cpp	everything described in this note
element_lagrangian_tangent	$\mathbb{A}$, eq. (23)
element_stiffness_from_ tangent	K^e , eq. (25)
FastStiffnessAssembler	cached pattern, parallel assembly of K^+ and of the Laplacian
include/optimization/ SparseCholesky.h, src/optimization/ SparseCholesky.cpp	SparseCholeskyBackend: CHOLMOD / Eigen factorization, half-solves and full solve
include/optimization/ StiffnessSpectrum.h, src/optimization/ StiffnessSpectrum.cpp	lowest_stiffness_modes: lowest eigenpairs of K (Section 12)
src/experiments/stability_ monitor.cpp	eigenvalues during the loading, flags <code>--eig-*</code>
transformed_energy	the ALGLIB callback (change of variables)
PreconditionedLBFGS::	setup, ALGLIB run, stopping, restarts
optimize	
BaseLatticeCalculator::	second derivatives without ITensor (+ overrides)
calculate_dseconderivative_ components	
energy_functions::create_ d2phi_components_func	the six analytic partials
relax_configuration() (common_ simulation_helpers.cpp)	single entry point used by the load steps, memory() and both remeshing loops; plain L-BFGS unless configured
main.cpp	flags <code>--precond=</code> , <code>--precond-tol=</code> , <code>--precond-refresh=</code> , <code>--precond-from-step=</code>
benchmarks/benchmark_ preconditioner.cpp	benchmark, <code>--validate</code> , <code>--eig N</code>
CMakeLists.txt, Makefile.cluster	CHOLMOD detection
makefiles_for_cluster/job_ creator*.py	preconditioner prompt (default <code>stiffness</code> , from step 1)

Usage. From the shell:

```
./lattice_triangulation 150 150 positive 42 1 --precond=stiffness --precond-from-step=1
./lattice_triangulation 150 150 positive 42 1      # original plain L-BFGS
```

Remeshing. The remeshing loop calls `relax_configuration()` too, so its relaxations are preconditioned as well, following the same `--precond-from-step` rule as the main solve of that load step. After each remesh the connectivity changes; the solver notices it (mesh signature) and rebuilds the pattern and the symbolic factorization, which costs a few tens of milliseconds.

Cluster builds. Without SuiteSparse the Eigen fallback is used ($\sim 8\times$ instead of $\sim 15\times$ at 150×150). `make -f Makefile.cluster info` tells which one is used. Because widely included headers changed (a new virtual method, a new member in `UserData`), the first rebuild must start from `make clean`; `Makefile.cluster` now tracks header dependencies, `Makefile.tchenla` does not yet.

15 Where the remaining time goes, and possible next steps

After this change the solve is no longer the only cost: in the running 150×150 simulation a load step takes ≈ 0.45 s in total, of which ≈ 0.30 s is the minimization. The rest is the energy/stress evaluations before and after, neighbour and defect analysis, remeshing, and output; before, it was hidden behind 2.9 s of L-BFGS.

Possible next steps:

- profile and trim the per-step overhead outside the solver (another factor ~ 1.5);
- decide whether instabilities without an accepted remeshing (such as step 1288 above) should count as avalanches;
- refresh K during long solves (every N iterations) if avalanche steps ever become the bottleneck;
- statistical comparison of avalanche distributions with and without preconditioner over several seeds, before using the preconditioned runs for publication-level statistics.

References

- [1] J. Nocedal, *Updating quasi-Newton matrices with limited storage*, Math. Comp. **35** (1980) 773–782.
The original limited-memory BFGS update and the two-loop recursion. The recursion takes an arbitrary initial matrix H_0 ; everything in Section 5 relies on this.
- [2] D. C. Liu and J. Nocedal, *On the limited memory BFGS method for large scale optimization*, Math. Programming **45** (1989) 503–528.
Convergence theory and the scaling $H_0 = \gamma_k I$, $\gamma_k = \mathbf{s}^T \mathbf{y} / \mathbf{y}^T \mathbf{y}$, that ALGLIB uses by default and that our change of variables turns into $H_0 = \gamma_k P^{-1}$.
- [3] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., Springer (2006), Chs. 6–7.
Textbook treatment of (L-)BFGS, preconditioning and its equivalence with a linear change of variables, and of Newton and trust-region methods, the natural competitors once K is available.
- [4] S. Bochkhanov, *ALGLIB numerical library*, <https://www.alglib.net>, manual of `minlbfgs` (version 3.19).
The optimizer we use. Its preconditioners (default, diagonal, dense Cholesky, scale) do not include a sparse factorization, which is why we precondition from outside (Section 5).
- [5] H. Matthies and G. Strang, *The solution of nonlinear finite element equations*, Int. J. Numer. Meth. Engng. **14** (1979) 1613–1626.
BFGS started from the factorized FE stiffness matrix, updated with low-rank corrections instead of refactorizing: the direct ancestor of what we do. The idea is standard in structural mechanics codes; our contribution is its application to a Lagrange-reduced, non-smooth energy.

- [6] L. Jiang, R. H. Byrd, E. Eskow and R. B. Schnabel, *A preconditioned L-BFGS algorithm with application to molecular energy minimization*, Tech. Rep. CU-CS-982-04, Univ. of Colorado (2004).
Preconditioned L-BFGS with a sparse preconditioner built from parts of the (partially separable) molecular energy; speedups 2–7 in CPU time for proteins. Same philosophy as ours: use the structure of the Hessian, not a generic diagonal.
- [7] D. Packwood, J. Kermode, L. Mones, N. Bernstein, J. Woolley, N. Gould, C. Ortner and G. Csányi, *A universal preconditioner for simulating condensed phase materials*, J. Chem. Phys. **144** (2016) 164109.
*The closest reference for the size-scaling argument of Section 4: for materials the condition number of the Hessian grows with system size, and a sparse Laplacian-type preconditioner built from the neighbour graph removes this growth. Implemented in ASE for L-BFGS and FIRE. Our **laplacian** option is the FE analogue; our **stiffness** option uses the exact Hessian.*
- [8] L. Mones, C. Ortner and G. Csányi, *Preconditioners for the geometry optimisation and saddle point search of molecular systems*, Sci. Rep. **8** (2018) 13991.
Sparse preconditioners built from the Hessian of force-field terms, keeping only the positive definite part: conceptually the same as our element-wise PSD projection (Section 7). They also use the preconditioner for saddle searches, a possible extension for avalanches.
- [9] T. Liu, S. Bouaziz and L. Kavan, *Quasi-Newton methods for real-time simulation of hyperelastic materials*, ACM Trans. Graph. **36** (2017) 23.
*L-BFGS for hyperelastic FE with a prefactorized, constant Laplacian-type matrix as initial Hessian; shows the large gains of a good H_0 in elasticity. Essentially our **laplacian** variant; our **stiffness** variant goes further by refactorizing the true tangent every step.*
- [10] J. Teran, E. Sifakis, G. Irving and R. Fedkiw, *Robust quasistatic finite elements and flesh simulation*, Proc. ACM SIGGRAPH/Eurographics Symp. Computer Animation (2005) 181–190.
Makes indefinite element tangents definite element by element (diagonalization and clamping), so that Newton steps are descent directions. Our PSD projection of the 4×4 tangent $\mathbb{A}$ is the same idea.
- [11] E. Bitzek, P. Koskinen, F. Gähler, M. Moseler and P. Gumbsch, *Structural relaxation made simple*, Phys. Rev. Lett. **97** (2006) 170201.
FIRE, the damped-dynamics minimizer widely used in athermal quasi-static simulations of plasticity and amorphous solids; the obvious baseline for any comparison beyond plain L-BFGS.
- [12] T. Ericsson and A. Ruhe, *The spectral transformation Lanczos method for the numerical solution of large sparse generalized symmetric eigenvalue problems*, Math. Comp. **35** (1980) 1251–1268.
Shift-invert (spectral transformation) Lanczos: eigenvalues near a shift σ become the dominant ones of $(K - \sigma I)^{-1}$. Basis of Section 12.
- [13] R. G. Grimes, J. G. Lewis and H. D. Simon, *A shifted block Lanczos algorithm for solving sparse symmetric generalized eigenproblems*, SIAM J. Matrix Anal. Appl. **15** (1994) 228–272.
The “industrial” shift-invert Lanczos of FE codes; uses the inertia of the LDL^T factorization of $K - \sigma I$ (Sylvester’s law) to count eigenvalues below a shift and to steer the shifts. Our shift search is the simplest version of this: a Cholesky failure means an eigenvalue below σ .
- [14] A. V. Knyazev, *Toward the optimal preconditioned eigensolver: locally optimal block preconditioned conjugate gradient method*, SIAM J. Sci. Comput. **23** (2001) 517–541.
LOBPCG: computes the lowest eigenpairs with a preconditioner instead of exact solves. A natural option for us, since the PSD-projected Cholesky factor is already available.

- [15] P. R. Amestoy, T. A. Davis and I. S. Duff, *An approximate minimum degree ordering algorithm*, SIAM J. Matrix Anal. Appl. **17** (1996) 886–905.
AMD, the fill-reducing ordering Π used before the Cholesky factorization (Section 8).
- [16] Y. Chen, T. A. Davis, W. W. Hager and S. Rajamanickam, *Algorithm 887: CHOLMOD, supernodal sparse Cholesky factorization and update/downdate*, ACM Trans. Math. Softw. **35** (2008) 22.
The supernodal sparse Cholesky we use (SuiteSparse). Its update/downdate feature could later allow modifying the factor after local changes instead of refactorizing.
- [17] Y. Qiu, *Spectra: C++ library for large scale eigenvalue problems*, <https://spectralib.org>.
Header-only implicitly restarted Lanczos/Arnoldi (ARPACK-like) used by the eigenvalue code; we give it our own Cholesky-based shift-invert operator.
- [18] S. Conti and G. Zanzotto, *A variational model for reconstructive phase transformations in crystals, and their relation to dislocations and plasticity*, Arch. Rational Mech. Anal. **173** (2004) 69–88.
The $GL(2, \mathbb{Z})$ -invariant energy and the Lagrange reduction used in this code.
- [19] R. Baggio, E. Arbib, P. Biscari, S. Conti, L. Truskinovsky, G. Zanzotto and O. U. Salman, *Landau-type theory of planar crystal plasticity*, Phys. Rev. Lett. **123** (2019) 205501.
The Landau-type model of crystal plasticity (Lagrange-reduced energy, quasi-static loading, avalanches) that the simulations in this note implement; the physics context of the solver.